BUILDBRIEF

Master Audit & Upgrade Execution Plan

31 October 2026 | Strategic synthesis v2.0

Preserve the foundation. Expand the intelligence. Prove the work.

Decisions locked: BOTH interaction modes · HYBRID execution · BOTH founders and developers

This document audits the supplied Build Brief v2 research, earlier product strategy and available implementation history. It distinguishes source-derived claims, recommendations and unresolved validation. It is an execution plan, not a claim that proposed features already exist.

1. Executive decision

BuildBrief should evolve into a **cross-agent development control and assurance system**. It should help users define what they intend to build, make that intent explicit and approved, carry it into the coding agent they choose, observe the work that the integration can genuinely see, test the result against executable and evidence-backed criteria, identify suspicious or unaccounted-for changes, coordinate a correction, and explain the outcome to both technical and non-technical people.

This is an evolution, not a rewrite. Preserve the current intent ledger, approval/versioning, repository evidence, verifier, correction packs, autopsy, understanding/quiz, MCP tools, CLI, demo fixtures and web application wherever they are sound. Refactor only where the audit proves a boundary is inadequate. Migrate data and APIs deliberately; never throw away working capabilities to obtain a cleaner architecture.

Locked product decisions
- Interaction: BOTH. A user can work from BuildBrief, or keep using an external coding agent with BuildBrief alongside it.
- Execution: HYBRID. Local-first repository access and checks, with optional cloud sync and hosted services later.
- Market: BOTH. Founders and developers use the same underlying intent/evidence truth, with different levels of explanation and control.

Product promise: “Build with AI. Stay in control.” More precise description: “BuildBrief keeps the approved intent, agent activity and implementation evidence connected, so you can see what changed, what is proven, what is uncertain and what needs attention.”

Do not promise that it catches literally everything or guarantees a perfect first attempt. Promise systematic, multi-layer detection with explicit coverage, traceable evidence, visible uncertainty and measured performance.

2. Audit of the supplied proposals: keep, adapt, defer, reject

KEEP — intent clarification before implementation. Ask a small number of high-value questions, prioritised by answerability, impact, dependency and cost of changing later. Recommendations need a plain-language reason and trade-off. Users must be able to answer, customise, defer or ask BuildBrief to choose a reversible default.

KEEP — owner-approved Intent Contract. AI extraction is a draft, not truth. Preserve raw request, extracted interpretation, assumptions, decisions, exclusions, acceptance criteria, provenance and owner approval as separate records. Changes create new versions. This directly addresses the user's observation that initial intent capture can be wrong.

KEEP — compile intent into executable acceptance checks. This is the strongest idea in the v2 research. Translate each requirement into one or more deterministic checks where feasible: tests, build, lint/typecheck, route/API smoke, schema, permission and protected-path checks. Keep semantic checks separate and use the model only where deterministic evidence cannot answer the question.

KEEP — agent hooks + MCP + local dashboard fallback. Use MCP for portable tools and structured workflows, hooks for lifecycle observation and limited enforcement, and a dashboard for a consistent visual surface. Offer three interaction tiers: native elicitation when supported; text-based questions through the agent; local BuildBrief UI link. Never assume all clients support native forms or inline apps.

KEEP — portable contract and evidence. Store the project contract and provenance in a repository-local `.buildbrief/` workspace or a safe external local workspace, then expose the same records to supported agents. Do not couple the contract to one agent transcript.

KEEP — tiered monitoring and stop-time verification. Deterministic checks can run after relevant edits or at turn end. Expensive semantic analysis should run at meaningful checkpoints, not on every keystroke. A capped correction loop is useful, but it must have a human escape hatch.

KEEP — A/B evaluation. Compare baseline agent workflow with BuildBrief-assisted workflow on the same tasks and same agent/model/configuration. Report turns to acceptance, constraint violations, unrequested changes, user clarification time, useful-alert rate, latency and cost. Report ties and losses honestly.

ADAPT — stop-gate. A hard gate can be valuable for clear, deterministic, high-confidence conditions, but it is unsafe to let an uncertain model verdict block every agent session. Use three dispositions: advisory warning; block only for explicit, owner-approved hard constraints or reproducible critical failures; human override with reason and audit record. After a small number of repair loops, stop automation and ask the owner. Never claim the agent cannot finish if the particular hook is advisory, unsupported or bypassable.

ADAPT — cross-agent portability. Start with two integrations, not three or every agent. The adapter contract must report capabilities per installed version. Muse Code, Codex and Claude Code have different hook events, transports, elicitation support and approval semantics. A provider matrix is a live test result, not a static marketing assumption.

ADAPT — comprehension map and quiz. Retain the existing understanding features, but only show grounded, source-linked explanations. Do not invent a score or imply a quiz proves deep comprehension. Offer short, optional learning checks and explain what evidence supports each answer.

DEFER — screenshot-to-visual-diff. Valuable for landing-page and UI work, but browser capture, reference alignment, responsive states and visual model interpretation add complexity. First support user-supplied reference images as explicit acceptance criteria, then compare screenshots as a clearly labelled advisory check.

DEFER — embedded coding agent as the first integration. Keep “inside BuildBrief” in the product vision, but start with the alongside workflow and a single embedded provider only when its supported SDK/App Server is stable for the target deployment. Recreating a terminal, editor, permission system and runtime would distract from BuildBrief's differentiation.

REJECT — fine-tuning a dedicated model as the solution. It is expensive, slow to validate and not needed to improve first-pass outcomes. Better structured context, targeted questions, acceptance checks, provenance and correction feedback are the practical levers.

REJECT — model judgement on every file edit. It is costly, slow, noisy and unnecessary. Use cheap local signals continuously; invoke semantic models at bounded checkpoints.

REJECT — “catch everything” as a literal guarantee. No product can infer all user intent, observe every hidden process, prove all runtime states or know every legitimate reason for a code change. BuildBrief should instead measure coverage and state the limits of observation.

3. The core product model

The product lifecycle becomes:

Discover → Clarify → Approve → Plan → Build → Observe → Verify → Correct → Re-verify → Understand → Release.

1. Discover: connect or create a project, identify stack, repository state, existing tests and agent.
2. Clarify: capture idea, user outcomes, constraints, assumptions, missing decisions and contradictions.
3. Approve: owner reviews a readable Intent Contract and versions it.
4. Plan: inspect the codebase and create implementation slices with acceptance criteria.
5. Build: hand the brief to the selected coding agent, either alongside or from BuildBrief.
6. Observe: collect supported session, tool, command, file and test events.
7. Verify: run deterministic checks, inspect diffs and use semantic analysis only for unresolved questions.
8. Correct: create a scoped evidence-backed correction, approved by the owner where required.
9. Re-verify: bind new results to the new commit/tree state and preserve history.
10. Understand: explain what changed and what is still unknown for each role.
11. Release: produce a release-readiness summary with verified checks and explicit gaps.

One shared truth, multiple views. Founder mode explains impact, decisions, risk and next action. Developer mode exposes criteria, file/symbol references, diffs, test logs and provenance. QA mode focuses on reproducibility and test evidence. Marketing/support mode sees approved product behaviour, release notes and limitations. Role views must not create separate conflicting requirements.

4. The “catch more” system: a layered detection architecture

BuildBrief's central improvement should be an **Assurance Pipeline**, not simply more model prompts. It should maintain a live, coverage-aware picture of the implementation.

Layer A — intent coverage. Every approved requirement has an ID, priority, type, owner, version, acceptance condition, dependencies and status. Detect missing acceptance criteria, conflicting requirements, unowned assumptions, unapproved changes and stale decisions. Require an owner to approve material interpretation changes.

Layer B — repository baseline. Capture repository root, branch, commit/tree hash, dirty state, detected stack, package manifests, entry points, routes, API handlers, database schema/migrations, test inventory, config, deployment manifests and relevant architecture. Respect ignore rules, symlinks, file-size limits and secret exclusions. Show inspected scope and exclusions.

Layer C — continuous change observation. Normalise provider events into a common event model: session started, prompt submitted, tool proposed, permission requested/decided, tool started/completed, file changed, command run, test completed, turn ended, session interrupted/ended. Each adapter declares which events it can capture and whether they are synchronous,

asynchronous or unavailable. Reconcile event stream with Git diffs because hooks may be missed or sessions may crash.

Layer D — change-impact mapping. For each changed hunk, classify likely relationship to approved criteria: directly implements; supporting change; test/documentation; unrelated/unclear; potentially conflicts; potentially violates protected path. Map file/symbol/test/API/schema dependencies. Flag **possibly unrequested changes**, never label them malicious or unauthorised without proof. Let the user explain legitimate refactors.

Layer E — deterministic checks. Run only checks relevant to the change and contract: formatting/lint, typecheck, unit/integration tests, build, API/schema compatibility, route smoke, migration validation, dependency/security audit, secret scan, licence/policy checks, protected-path rules, permission boundaries and selected browser workflows. Store command, environment, exit code, stdout/stderr digest, duration, revision and timestamp. A test passing on an old revision is stale evidence.

Layer F — semantic verification. Use a model only for criteria that cannot be decided deterministically: whether a feature behaviour appears implemented, whether a changed flow contradicts a user outcome, whether documentation and code disagree, or whether a proposed correction addresses the actual failure. Model output must be schema constrained, cited to retrieved evidence and independently validated. Model output is an assessment, not proof.

Layer G — runtime and visual evidence. Add local browser automation, API calls, staging checks, screenshots and telemetry only when explicitly configured and authorised. Distinguish local, test, staging and production. Do not infer production correctness from a local build. A screenshot comparison is advisory unless acceptance thresholds and capture conditions are defined.

Layer H — release and operational checks. Inspect configuration completeness, environment variable references (not values), migrations, dependency vulnerabilities, observability, rollback notes, error handling, accessibility, performance budgets and deployment readiness. These are separate from feature-intent compliance.

Coverage and uncertainty. Each report must show: criteria inspected / total; files and directories inspected; events captured / expected where measurable; checks run / skipped; stale evidence; unsupported surfaces; time window; commit/tree hash; confidence and reason. “Not found” is allowed only after adequate declared search. If search is incomplete, say “not established”. If evidence conflicts, say “conflicting evidence”. Never turn absence of evidence into evidence of absence.

Detection catalogue. Include at least: missing behaviour; partial implementation; contradictory behaviour; broken tests/build; stale evidence; unapproved intent drift; suspicious/unrelated diffs; protected-path edits; deleted or weakened tests; API/schema breaking changes; security-sensitive changes; secrets exposure; dependency risks; unsafe file operations; accessibility gaps; runtime failures; missing error/loading/empty states; performance regressions; data migration risks; missing docs/release notes; unobserved agent actions. Each detector needs severity, trigger, evidence requirement, false-positive handling and remediation path.

5. Product experience and landing page redesign

The current product experience should stop looking like an internal engineering console. Preserve the underlying data and capabilities, but rebuild the information architecture and language around a user's project and next decision.

Landing page goal: explain the product in five seconds. It should not lead with “intent ledger”, “verification engine” or jargon. Hero direction:

Headline: **“Your AI coding agent builds. BuildBrief keeps it accountable.”**
Subheading: “Turn an idea into approved requirements, guide your coding agent with the right context, and see what was actually built, what passed, and what still needs attention.”
Primary CTA: “Start a project”. Secondary CTA: “See how it works”.
Visual: a live, understandable project flow showing an approved requirement connected to a code change, test evidence and a correction. Avoid generic robot imagery or fake dashboards.

Landing page sections
1. Hero with real product UI and one clear outcome.
2. Problem: repeated prompting, misunderstood requirements, hidden side effects, unclear “done”.
3. How it works: clarify, approve, build, observe, verify, correct.
4. Product demonstration with real or explicitly labelled sample data.
5. Works alongside supported coding agents; inside-BuildBrief mode where supported.
6. Founder and developer benefits, without splitting the product promise.
7. Trust section: evidence provenance, local-first option, transparent capture gaps, human approval.
8. Use cases: greenfield feature, existing-codebase change, bug fix, UI implementation, team handover.
9. Pricing/plan only when unit economics and access model are real.
10. FAQ: what BuildBrief can observe, what it cannot, local/cloud privacy, agent compatibility.
11. CTA and footer.

Authenticated application IA
- Home: projects, active sessions, attention queue, latest verification and next action.

- Project Room: overview, active task, contract progress, agent connection, activity, preview and recent evidence.
- Intent: raw request, questions, recommendations, assumptions, exclusions, approved versions and change history.
- Plan: implementation slices, dependency map and acceptance checks.
- Live Build: selected agent, session state, event timeline, permission requests, changed files and captured gaps.
- Assurance: criteria matrix, checks, evidence, coverage, freshness and risk.
- Changes: diff review, possible drift, protected-path changes, disposition and explanation.
- Fixes: correction pack, approval, agent handoff, loop count and re-verification.
- Understand: role-based explanation, grounded Q&A, optional quiz.
- Connections: agent capability matrix, local daemon status, permissions, privacy and budgets.

Visual rules: restrained, premium, professional and clear. Strong typographic hierarchy; generous spacing; consistent grid; deliberate colour for status only; polished skeleton/loading/empty/error states; responsive layout; keyboard support; reduced motion. Use animation to explain state transitions and relationships. Use 2D dependency/evidence maps when they help comprehension. 3D is optional marketing or onboarding, not a prerequisite for a usable product.

Language rules: replace raw enums and identifiers with “Partly implemented”, “We have not confirmed this yet”, “This check has not run on the latest change”, “This file changed but is not linked to an approved requirement”. Put technical detail behind “View evidence”. Never hide severity, confidence or limitations.

6. Adaptive clarification and planning engine

Use an adaptive question graph rather than a static questionnaire. For each candidate question estimate: expected impact on architecture/acceptance, uncertainty, answerability for this user, reversibility, dependencies, cost of later change and whether the repository already answers it. Ask the highest-value question first. Aim for a short first pass; do not impose an arbitrary fixed number when a critical decision remains unresolved.

Question layers
- Product facts: target user, job, primary outcome, workflow, payment/currency/data rules.
- Scope boundaries: in/out, protected areas, integrations, permissions, compliance and non-goals.
- Quality: acceptance scenarios, accessibility, performance, reliability and definition of done.
- Technical choices: only after inspecting repository; recommend existing stack and patterns unless there is a strong reason to change.

Every question has: why it matters; recommended answer; one-line trade-off; alternatives; custom response; “not sure”; “decide later”; “let BuildBrief choose” only for reversible decisions. Technical choices should not be pushed onto a non-technical founder.

Contract output
- problem and target user
- goals / outcomes
- functional requirements
- non-functional requirements
- constraints and protected paths
- non-goals
- assumptions (with source and owner)
- decisions and unresolved questions
- acceptance criteria and executable checks
- dependencies and risks
- owner, version, timestamp and approval record

Plan compiler: break approved intent into small implementation slices. Each slice has prerequisite, files/modules likely affected, expected behaviour, acceptance checks, risk and rollback considerations. It should use repository facts rather than invent architecture.

7. Hybrid architecture and agent integration

Recommended topology

Local BuildBrief daemon (per workspace) owns repository access, Git snapshots, local event intake, deterministic checks, SQLite/JSONL ledger, secret redaction and local UI/API. Agent adapters connect through MCP, hooks, CLI or provider SDK. A browser dashboard consumes a local event stream (SSE/WebSocket where appropriate). Optional cloud control plane later provides encrypted sync, user/team accounts, notifications, hosted jobs and billing.

Domain packages / services
1. Intent and approval service.

2. Clarification/question planner.
3. Repository context/index service.
4. Plan and acceptance-check compiler.
5. Agent adapter registry and capability negotiation.
6. Event normaliser and session correlator.
7. Diff/change-impact mapper.
8. Deterministic check runner and sandbox policy.
9. Evidence store and verifier.
10. Correction/autopsy service.
11. Role explanation and learning service.
12. Evaluation, budget and audit telemetry.

Adapter interface (conceptual)
- detect/install/configure/uninstall and health check
- provider name, version and capability manifest
- start/resume/interrupt session if supported
- send prompt/context/steering message if supported
- subscribe to events and report capture gaps
- request/resolve approvals only where provider supports it
- retrieve transcript/session identifiers where permitted
- map provider-specific file/tool/command events to common events
- report whether enforcement is advisory, blocking or unsupported

Do not pretend all adapters can steer, interrupt, approve or block. Capabilities are negotiated and surfaced to the user.

Integration order
1. Harden existing MCP server and local CLI.
2. Add the first hook adapter for Muse Code, after verifying the installed version's actual event schema and permissions.
3. Add Codex hooks for alongside monitoring; use Codex SDK for local programmatic jobs and App Server for a deep embedded client only after validating stability, auth and production support.
4. Add Claude Code after independent hook/MCP validation.
5. Expand based on demand, not a long compatibility checklist.

Elicitation fallback: native MCP elicitation when the host supports it; otherwise return numbered questions and collect answers through the agent; otherwise open the local BuildBrief questionnaire. The MCP protocol standardises request/response, not the exact UI. Never request secrets through elicitation forms.

Security boundary: local-first means source stays local unless the user explicitly opts in. Cloud upload is per-project and field-level visible. Treat repository files, comments and tool output as untrusted data, not privileged instructions. Require user approval for destructive actions, external network calls, protected paths and high-impact corrections.

8. Data, verification and trust contracts

Core entities: Project; RepositoryConnection; Intent; immutable IntentVersion; Decision; ApprovalEvent; Plan; TaskSlice; AcceptanceCheck; AgentInstallation; AgentSession; AgentEvent; RepositorySnapshot; DiffHunk; EvidenceItem; CheckExecution; VerificationRun; Finding; FindingDisposition; CorrectionPack; LearningActivity; BudgetEvent.

Revision identity: every finding references repository ID, commit SHA or tree hash, dirty-worktree digest where needed, intent version, evidence IDs, check-run IDs, timestamps, adapter version and verifier/prompt/schema version. If the working tree changes, evidence that depends on it becomes stale.

Finding states: supported; partially supported; not found within inspected scope; contradicted; unknown/not established; conflicting. Keep implementation state separate from evidence grade, coverage, freshness, severity and owner disposition.

Evidence levels
- E0 no usable evidence
- E1 repository structure/symbol discovery
- E2 source evidence pinned to revision and exact path/range/blob
- E3 test/build/lint evidence pinned to revision and execution environment
- E4 authorised local/staging runtime observation
- E5 production telemetry with defined window and source

A higher evidence level is not automatically stronger for every criterion. For example, a passing unit test may not prove production behaviour.

Verifier pipeline
1. Load approved intent version and exact repository snapshot.
2. Resolve each criterion to likely files, symbols, routes, tests and dependencies.
3. Run deterministic checks and capture outputs.
4. Retrieve bounded source/diff context.
5. Ask model only for unresolved semantic questions.
6. Require structured output with criterion ID, verdict, confidence rationale, evidence references, coverage and limitations.
7. Mechanically validate each path, line, hash and test ID; reject fabricated citations.
8. Reconcile with test/runtime evidence and mark stale/unknown/conflicting.
9. Persist immutable run; compare against prior revision.
10. Present plain-language summary and detailed evidence.

Anti-false-positive rules
- A deleted test is a finding; it does not prove the feature is broken until impact is assessed.
- A diff unrelated to explicit criteria is “possibly unrequested”, not automatically a violation.
- A requirement not found in searched files is “not established” if search coverage is insufficient.
- Comments and docs do not prove implementation.
- Passing tests prove only the tested conditions.
- Model confidence is not evidence.
- An empty commit does not prove a correction. A meaningful changed hunk plus relevant rerun is required.
- A model cannot silently approve its own extracted intent or its own correction.

9. Security, privacy, resilience and budget

Security controls
- local daemon binds loopback by default; no unauthenticated network exposure
- authenticated cloud API, tenant isolation, least privilege and short-lived credentials
- secrets never enter repository, prompts, logs, screenshots, URLs or exported ledgers
- OS credential store or managed secret service for tokens
- repository ignore rules, secret scanning, file-size limits, archive limits, symlink/path traversal protection
- safe process execution: allowlisted commands, isolated worktree/container, timeout, resource limits, explicit network permissions
- protect against prompt injection in code/docs/issues and malicious MCP tool output
- explicit approvals for writes, deletion, deployment, database migration and external side effects
- audit log for approvals, overrides, exports, connections and cloud transfers
- retention, export and deletion policy; visible local/cloud data boundary
- tests for cross-project leakage, path escape, symlink escape, malicious filenames and credential redaction

Reliability: queue and retry events idempotently; handle duplicate/out-of-order hooks; persist before acknowledging; recover after daemon restart; reconcile missed events against Git; show adapter heartbeat and gaps. A lost hook must never be rendered as “no changes”.

Token/cost controls
- deterministic-first; model only for semantic ambiguity
- retrieve relevant files, not whole repositories
- cache by project, intent version, commit/tree hash, file hashes, check IDs, model, prompt and schema version
- invalidate cache on dependency changes
- strict input/output limits and truncation warnings
- per-user/project daily/monthly caps, per-run estimate, retry cap and hard stop
- no paid calls in unit tests/CI; fixtures for deterministic tests
- one authorised live smoke test per new provider path
- record tokens, cache hits, latency, retries and cost per useful task
- distinguish BuildBrief inference cost from coding-agent subscriptions and cloud infrastructure

Initial alpha guardrail: begin with a low monthly BuildBrief inference cap (for example US$25 for a private alpha), measure real cost per successful task, then adjust. This is a proposed budget control, not a forecast of how many users or requests it buys.

10. Phased implementation plan — preserve, then extend

Phase 0 — Baseline audit and freeze (P0). Inspect the live repository and current docs; reconcile M4e/M5/M5c claims with actual owner-side browser run and latest code; verify current branch/commit/dirty state; run targeted tests; list every current capability as working, partial, broken or untested. Resolve any stale demo-only assumptions. No redesign or new feature until the baseline is signed off.

Phase 1 — Domain contracts and assurance foundation (P0). Preserve existing ledger and verifier; add/confirm immutable intent versions, approval provenance, revision identity, evidence coverage/freshness, event capture gaps and finding disposition. Add migrations and compatibility tests. No destructive schema replacement.

Phase 2 — Project Room and landing experience (P0). Redesign landing page and app navigation; build Home, Project Room, Intent, Assurance and Changes screens on existing APIs first. Add founder/developer modes, plain language, onboarding and all empty/loading/error states. Visual polish must not hide missing evidence.

Phase 3 — Adaptive intake and plan compiler (P0). Build question prioritisation, recommendation rationale, answer/defer/custom flows, contract review/approval and implementation slices. Use repository facts to avoid asking questions already answered.

Phase 4 — Alongside-agent integration (P0/P1). Formalise adapter interface and capability negotiation. Harden MCP/CLI; implement Muse Code hook adapter first, then Codex or Claude Code as second integration. Add local dashboard fallback. Validate real events and event loss/recovery.

Phase 5 — Assurance pipeline (P0). Change-impact mapping, deterministic check compiler, selective test runner, security/dependency checks, semantic verifier at turn end, mechanically validated citations, evidence coverage and stale detection. Build benchmark cases for false positives and false negatives.

Phase 6 — Correction and controlled gate (P1). Scoped correction packs, owner approval, bounded repair attempts, high-confidence deterministic block rules, advisory semantic warnings, override reasons and fresh revision re-check. Never use a model-only block as a hard stop by default.

Phase 7 — Understanding and release readiness (P1). Grounded role-based explanations, optional quiz, release checklist, known limitations and exportable evidence report.

Phase 8 — Embedded agent and cloud (P2). Add one embedded SDK/App Server workflow where stable; optional cloud sync/accounts/team collaboration only after threat model, authentication, tenant isolation and economics are proven.

Definition of done for every phase: acceptance criteria written first; migration plan; positive and negative tests; permission/security checks; error/timeout/offline paths; browser validation; documentation; before/after screenshots; actual test report; no unsupported success claims.

11. Priority backlog and acceptance criteria

P0-A Baseline and preservation. Inventory every current feature, API, package and test. Acceptance: capability matrix includes evidence and owner confirmation; no working capability silently removed.

P0-B Intent truth model. Immutable versions, raw request, AI draft, user edits, owner approval and audit trail. Acceptance: model cannot approve its own interpretation; amendments create new version; old runs remain readable.

P0-C Revision-aware evidence. Pin evidence to commit/tree/worktree digest, test run and time. Acceptance: any relevant changed file invalidates dependent evidence; stale findings are obvious.

P0-D Assurance coverage. Per-criterion coverage, inspected scope, checks run/skipped, unsupported integration events and confidence reason. Acceptance: incomplete scan cannot output definitive “not implemented”.

P0-E Adaptive intake. Question value ranking, recommendation, trade-off, custom answer, defer and repository-aware skip. Acceptance: 5 greenfield and 5 existing-repo scenarios tested with founders and developers.

P0-F Project Room UX. Landing, onboarding, home, project, intent, activity and assurance. Acceptance: first-time founder can start and understand next action; developer can inspect evidence within two clicks.

P0-G Agent adapter framework. Versioned capability manifest, event normalisation, health, permissions and fallback. Acceptance: unsupported event never silently appears as captured.

P0-H Deterministic check compiler. Map criteria to commands and check definitions; safe execution policy. Acceptance: checks are reproducible, revision-pinned and scoped; timeout/failure/skipped distinguished.

P1-I Change-impact and drift. Diff hunks linked to criteria, supporting changes and possible orphans. Acceptance: benchmark includes legitimate refactors and intentionally unrelated changes; report precision and false alarm rate.

P1-J Stop-gate/correction loop. Bounded loops and human override. Acceptance: deterministic hard constraints can block; semantic uncertainty warns; max loops enforced; interruption/recovery works.

P1-K Runtime/visual evidence. Browser/API/staging integration with environment labels. Acceptance: no local result described as production proof; screenshot comparisons repeatable and advisory.

P1-L Role explanations and learning. Grounded explanation citations and optional assessment. Acceptance: quiz answers cite current source; no unsupported mastery score.

P2-M Cloud/team/embedded agent. Auth, sync, RBAC, tenant isolation and one deep embedded adapter. Acceptance: security review, deletion/export tests, threat model, cost model and recovery plan.

12. Evaluation plan: prove the system catches more

Build a labelled benchmark with at least these case families: correctly implemented; partially implemented; missing with adequate search; missing with inadequate search; explicit contradiction; ambiguous requirement; misleading comment; fake citation; stale source evidence; stale test run; legitimate refactor; unrequested feature; deleted test; weakened assertion; protected-path edit; secret introduced; dependency vulnerability; API breaking change; migration hazard; event hook missed; interrupted session; failing runtime; accessibility regression; visual mismatch.

For each case record ground truth, required evidence, acceptable verdicts, severity and whether abstention is correct. Use separate development and holdout sets, independent review and no hand-editing metrics.

Metrics
- criterion-level precision, recall and F1
- false-positive rate and false-negative rate by detector
- correct abstention rate on insufficient evidence
- citation validity and revision freshness
- requirement coverage and event capture coverage
- time from change to useful detection
- useful-alert / dismissed-alert ratio
- correction success rate after fresh re-verification
- turns to acceptance and constraint violations
- founder understanding and developer trust
- model tokens, latency and cost per useful finding

A/B study: same tasks, same agent/model/version, same environment, randomised order where practical. Condition A: baseline agent. Condition B: BuildBrief clarification + contract + checks + observation. Use 5–10 tasks for an initial directional test; larger samples needed for robust claims. Report all results and limitations. Do not use one seed demo or one reviewer to claim real-world accuracy.

User tests: 5–10 founders and 3–5 developers. Watch them complete an idea-to-approved-brief task and an existing-repo change. Measure time, confusion, question quality, evidence comprehension, false-alarm tolerance and task completion. Interview users after the task, not only from stated preferences.

13. Budget framework and delivery sequence

The prior v1 strategy budget and the v2 four-day hackathon sprint are not interchangeable. The v2 PDF's US$90–150 figure is a short hackathon model/inference allowance, not the cost of building the commercial product. Likewise, earlier MVP engineering ranges are planning estimates, not vendor quotes. Re-estimate from actual work packages and local quotes before committing funds.

Budget categories to track separately
1. Product/UX design and user research.
2. Engineering by work package (domain, adapters, assurance, web UX, cloud).
3. QA, security review and browser/device testing.
4. Hosting, database, storage, backups, monitoring and email.
5. BuildBrief inference usage.
6. Coding-agent subscription/credits (separate from BuildBrief API usage).
7. Legal, privacy, tax, payment and compliance work.
8. Contingency and maintenance.

Cost strategy: build local-first; keep source analysis local; use the model only for high-value semantic checks; cap loops; run deterministic checks on every change; cache revision-specific context; use sample fixtures offline; defer cloud and broad integration work. The highest hidden cost is not model tokens alone: it is maintaining adapters as providers change their hook contracts.

Suggested sequencing: deliver Phases 0–5 before spending heavily on embedded runtime or cloud. The first commercially credible milestone is not “all agents supported”; it is one excellent end-to-end workflow, two tested adapters, honest evidence coverage, low false-positive rate and a UI users understand.

14. Risks and explicit non-goals

Key risks
- Hook/API changes break adapters: version capability manifests and contract tests.
- Events are lost or out of order: idempotency, durable queue and Git reconciliation.
- Model invents proof: citation validation, deterministic checks and abstention.
- Hard gates frustrate users: only deterministic high-confidence blocking, bounded retries and override.
- More alerts create noise: severity thresholds, suppression and measured useful-alert ratio.
- Broad scope delays quality: phase gates and explicit cut lines.
- Cloud creates security liability: local-first until auth/tenant controls are ready.
- Founder UX becomes too technical: plain-language default, details on demand.
- Developer UX becomes too simplistic: exact evidence and provenance always accessible.
- Competitors overlap: differentiate on end-to-end evidence continuity and cross-agent intent control, not generic spec generation.

Non-goals for this upgrade
- Replacing Codex, Muse Code, Claude Code or other coding agents.
- Building a full IDE/editor/terminal from scratch.
- Fine-tuning a proprietary model as the first solution.
- Blocking every tool call or judging every file edit with an LLM.
- Guaranteeing perfect first-pass builds or complete detection.
- Launching a native mobile app before core desktop workflows work.
- Adding 3D motion before onboarding and evidence UX are excellent.
- Exposing local repository access over a public network without authentication.

15. Master execution prompt for the coding agent

You are the principal engineer, product designer and assurance architect responsible for upgrading the existing BuildBrief application. This is a major evolution of an existing product, NOT a greenfield rewrite.

Read this plan and the repository's current AGENTS.md, README, MASTER_BLUEPRINT, M4e/M5/M5c audit reports, progress/decision/known-issue docs and current source before making changes. The live repository is the authority for what exists. Historical reports are evidence to re-check, not proof that the current working tree is unchanged.

LOCKED DECISIONS: both interaction modes (inside BuildBrief and alongside existing agents); hybrid local-first plus optional cloud; both founders and developers with role-adapted views.

NON-NEGOTIABLES:
1. Preserve all working capabilities and existing data. No wholesale rewrite.
2. First produce a baseline inventory: working, partial, broken, untested, security-sensitive and owner-dependent.
3. Reconcile reported browser results and latest genuine-fix E2E results; do not conflate test discovery with browser execution.
4. Map every proposed change to existing packages/routes/APIs and identify migration needs.
5. Use additive, backwards-compatible migrations. Never silently delete or replace working data.
6. Owner approval is the authority for intent. AI extraction and model judgement are drafts.
7. Every finding must be revision-pinned, evidence-backed and coverage-aware.
8. Deterministic checks first. Model calls only for semantic uncertainty, with strict budget controls.
9. Hooks and adapters report actual capabilities and capture gaps. Never claim universal real-time monitoring.
10. Hard-block only explicit owner-approved constraints or reproducible critical deterministic failures. Semantic model findings are advisory by default. Provide human override and audit record.
11. A correction is not verified until meaningful code changes and relevant checks are run on the new revision.
12. Local-first privacy. No secrets in source, logs, URLs, prompts or exports. No unauthenticated network exposure.
13. Do not stage generated files, secrets, caches, ledgers or scratch work. Never use blind `git add -A` where repository ownership or ignored artifacts are uncertain.
14. Never fabricate tests, metrics, provider compatibility, runtime proof or completion claims.

FIRST RESPONSE BEFORE CODING:
A. Repository inventory and exact current Git state (if available).
B. Existing architecture map and capability matrix.
C. Reconciliation of historical M4e/M5/M5c evidence against current code.
D. Risk register and security boundary.
E. Proposed additive architecture and migration plan.
F. Dependency-ordered work packages with file ownership.
G. Tests and acceptance criteria per package.
H. Estimated effort and external dependencies.
I. Questions that genuinely block progress; use reversible defaults for the rest.

IMPLEMENTATION ORDER:
Phase 0 baseline audit and preserve.
Phase 1 intent approval, revision and evidence contracts.
Phase 2 Project Room / landing / founder-developer UX.
Phase 3 adaptive intake and plan compiler.
Phase 4 local MCP/CLI plus first two agent adapters.
Phase 5 assurance pipeline and coverage.
Phase 6 correction and safe stop-gate.
Phase 7 role explanations and release readiness.
Phase 8 embedded agent/cloud only after explicit gate approval.

TOKEN AND COST DISCIPLINE:
- Inspect targeted files and reuse existing docs before searching broadly.
- Do not repeat successful full suites without a code or environment change.
- Use fixtures for routine tests; no paid inference in CI.
- Estimate and request approval for expensive live calls.
- Cache by intent version, repo revision, selected file hashes, checks, model and prompt/schema version.
- Set hard per-project budgets and retry limits.
- Report actual usage and cost.

QUALITY GATES:
- Every epic starts with acceptance criteria and a test plan.
- Run focused tests after each change; full suite only at milestone gates.
- Include negative, adversarial and insufficient-evidence cases.
- Validate UI in a real browser at desktop and mobile widths.
- Record screenshots and console/network errors.
- Keep docs and progress records current.
- Commit in small, reviewable changes; show exact staged files before owner commit.

Do not begin by implementing every feature. Complete the audit and plan first. Then implement the highest-priority phase incrementally, stop at each gate, and show evidence. The goal is a trustworthy product, not a large feature count.

16. Research references and source limits

• **Muse Code extension mechanisms**: https://meta-models.github.io/muse-code-sdk/next/guides/extend/

• **Muse Code SDK overview**: https://meta-models.github.io/muse-code-sdk/

• **Muse Code product and API distinction**: https://dev.meta.ai/docs/muse-code

• **Codex App Server**: https://learn.chatgpt.com/docs/app-server

• **Codex SDK**: https://learn.chatgpt.com/docs/codex-sdk

• **Codex hooks**: https://learn.chatgpt.com/docs/hooks

• **MCP elicitation specification**: https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/client/elicitation.mdx

• **GitHub Spec Kit**: https://github.github.io/spec-kit/

• **OpenAI developer docs MCP**: https://developers.openai.com/learn/docs-mcp

Source-derived findings are limited to the attached documents and current official documentation. Third-party capability matrices and research summaries in the supplied v2 PDF are not treated as authoritative without primary-source confirmation. Research statistics should not be used in marketing until the original papers and applicability are independently checked. Prior budget figures are estimates, not quotes.

17. Final recommendation

Approve this as the master direction, but execute it as gated upgrades. The first engineering task is a repository baseline and preservation audit, not immediate feature implementation. The first product-facing milestone should combine a redesigned Project Room, approved adaptive intent and a coverage-aware assurance report on top of the existing engine. The strongest differentiator is connecting approved intent to actual changes and verifiable evidence, exposing blind spots honestly, and helping users correct the work.